Project Report

Submitted by:

Name: Ali Hussain

ID: 231-115-170

Name: Marjia Chowdhury

ID: 231-115-183

Batch: 58th

Section: E

Course Name: Compiler Construction Lab

Course Code: CSE 416

Department of Computer Science and Engineering

Submitted to:

Mahbuba Akther Liza

Lecturer

Department of Computer Science and Engineering

Metropolitan University, Sylhet.

Table of Contents

1. Introduction
    1.1 Project Background
    1.2 Motivation
2. Objectives
3. Language Specification
    3.1 Supported Language Features
    3.2 Common Features
    3.3 Lexical Elements
    3.5 Formal Context-Free Grammar (CFG)
    3.5.1 C CFG
    3.5.2 C++ CFG
    3.5.3 Java CFG
4. Compiler Architecture
    4.1 Module Communication
    4.2 Driver Control
    4.3 Frontend and Backend Support
5. Lexer Design
    5.1 Token Categories
    5.2 Regular Expressions Used in the Lexer
    5.3 Important Flex Decisions
    5.4 Ignored Text and Errors
6. Parser Design
    6.1 Grammar Used in the Parser
    6.1.1 Statement Grammar
    6.1.2 Conditional Statements
    6.2 Expression Grammar
    6.3 Precedence and Associativity Rules
    6.4 Conflict Resolution
    6.5 AST Construction
    6.6 Parser Error Handling

7. Abstract Syntax Tree (AST)
    7.1 Tagged ASTNode Design
    7.2 AST Construction Strategy
8. Semantic Analysis
9. Symbol Table
    9.1 Data Structure Design
    9.2 Scope-Handling Strategy
10. Intermediate Code Generation
    10.1 Control Flow
11. Challenges
12. Testing
    12.1 C Language – Lexical, Syntax, Semantic Error Examples
    12.2 CPP Language – Lexical, Syntax, Semantic Error Examples
    12.3 Java Language – Lexical, Syntax, Semantic Error Examples
    12.4 Valid Program Example
13. Conclusion
    13.1 Limitations
14. References

1. Introduction

Compiler construction is the study of how a program written in a human-readable programming language is processed into a form that a computer system can execute. A compiler is not one single algorithm. It is a sequence of cooperating phases. Each phase receives a structured result from the previous phase, performs one clearly defined task, and passes its result forward. This project implements those ideas in a working educational compiler system built around Flex and Bison. The project is organized as three closely related compiler implementations: C_Compiler, CPP_Compiler, and Java_Compiler. Each implementation has its own lexer.l and parser.y files, while the remaining compiler modules follow the same overall design: AST construction, symbol-table handling, semantic analysis, interpretation, and Three Address Code generation. The web interface and Node.js server select one of these compiler modules.

1.1 Project Background

The compiler front end begins with raw source text. The lexer recognizes characters and groups them into tokens. Bison then checks the token sequence against a grammar and constructs an Abstract Syntax Tree. The semantic analyzer walks that tree and uses a symbol table to detect errors that syntax alone cannot detect. The interpreter later walks the validated tree to execute the program. A TAC generator also walks the tree to produce an intermediate representation for inspection.

1.2 Motivation

- To understand how a CFG controls parsing and how parser actions create an AST.
- To understand how semantic analysis uses a symbol table rather than relying only on grammar rules.
- To demonstrate how expressions and control flow can be represented as TAC using temporaries and labels.

2. Objectives

The main objective is to build an educational compiler system in which the major compiler phases communicate through explicit intermediate data structures.

Objective	**How it is demonstrated**
Lexical analysis	Flex rules recognize keywords, identifiers, literals, operators, delimiters, comments, and language-specific tokens.

Syntax analysis	Bison grammars validate the source structure and construct AST nodes during reductions.
AST generation	ast.h/ast.cpp define one tagged ASTNode structure used by later phases.
Semantic analysis	semantic.cpp walks the AST and checks declarations, assignments, constants/finals, division by zero, control-flow restrictions, and language-specific invalid uses.
Symbol table	symbol_table.cpp stores names, types, scopes, values, and const/final information.
CFG implementation	Each parser.y is a formal grammar implemented in Bison, with precedence declarations for expressions.
TAC generation	tac.cpp converts many AST constructs into temporaries, assignments, labels, conditional jumps, and print operations.
Execution/interpreter	interpreter.cpp evaluates the AST directly and produces the actual program output.
Error detection	The system reports lexical, syntax, semantic, and runtime errors at different stages.
Debug visibility	--debug exposes tokens, AST, semantic status, TAC, pseudo code, pseudo assembly, output, and the final symbol table.

3. Language Specification

3.1 Supported Language Features

The following feature list is derived from the lexer rules, Bison grammars, AST node types, semantic analyzers, interpreters, TAC generators and sample programs.

Feature	**C compiler**	**C++ compiler**	**Java compiler**
int / integer literals	Yes	Yes	Yes
float, double	Yes	Yes	Yes
char	Yes	Yes	Yes
boolean type	No dedicated bool type	Yes: bool	Yes: boolean
string type	String literals supported for printf/TAC	Yes: string	Yes: String
const/final	const	const	final

Declaration with initializer	Yes	Yes	Yes
Multiple declarators in one statement	Yes	Yes	No
Simple assignment	Yes	Yes	Yes
Compound assignment	+= -= *= /= %=	+= -= *= /= %=	+= -= *= /= %=
Increment/decrement	Prefix and postfix	Prefix and postfix	Prefix and postfix
Arithmetic	+ - * / %	+ - * / %	+ - * / %
Relational	< > <= >= == !=	< > <= >= == !=	< > <= >= == !=
Logical	&& || !	&& || !	&& || !
Bitwise / shifts	& | ^ ~ << >>	& | ^ ~ << >>	& | ^ ~ << >>
Ternary ?:	Yes	Yes	Yes
if / else / else-if	Yes	Yes	Yes
while	Yes	Yes	Yes
do-while	Yes	Yes	Yes
for	Yes	Yes	Yes
switch / case / default	Yes	Yes	Yes
break / continue	Yes	Yes	Yes
return	Yes	Yes	Yes
Nested blocks	Parsed and traversed	Parsed and traversed	Parsed and traversed
C-style printf/scanf	Yes	No	No
C++ cout/cin/endl	No	Yes	No
Java System.out / Scanner	No	No	Yes
sizeof	Yes	Yes	No
Built-in math functions	sqrt, pow, abs, ceil, floor	No dedicated math built-ins	No dedicated math built-ins
Imports / includes	#include lines ignored	#include and using namespace std ignored	import statements retained as AST no-ops
Arrays / pointers / structs / classes	No	No	No user-defined classes
User-defined functions	No; main only	No; main only	No; main only

3.2 Common Features

The three compiler modes share a large core. All three recognize identifiers, numeric literals, character literals, strings, arithmetic expressions, comparisons, logical expressions, control-flow statements, assignment, compound assignment, increment/decrement, blocks, and return statements. This common structure is reflected in similar AST node types such as NODE_PROGRAM, NODE_BLOCK, NODE_VARDECL, NODE_ASSIGN, NODE_COMPOUND_ASSIGN, NODE_BINOP, NODE_UNOP, NODE_IF, NODE_WHILE, NODE_FOR, NODE_BREAK, NODE_CONTINUE, and NODE_RETURN.

3.3 Lexical Elements

Category	Implementation
Identifiers	Start with a letter or underscore, followed by letters, digits, or underscores.
Integer literals	Recognized by digit sequences.
Floating literals	Recognized by digit-dot-digit patterns; Java also accepts f/F/d/D suffixes.
Character literals	Single quoted character forms with basic escape handling.
String literals	Double quoted strings with basic escape handling.
Comments	Single-line // and multi-line /* ... */ comments are discarded.
Whitespace	Spaces, tabs, carriage returns, and newlines are ignored.
C/C++ preprocessing	#... lines are skipped by the lexer; C++ also skips using namespace std;.
Java compound tokens	System.out.print, System.out.println, and System.in are recognized directly.

3.5 Formal Context-Free Grammar (CFG)

A Context-Free Grammar is a set of production rules that describes which token sequences form valid programs.

3.5.1 C CFG:

```
<program> ::= int <id> ( ) <block>
<block> ::= { <stmt_list> }
<stmt_list> ::= ε | <stmt_list> <stmt>
<stmt> ::= <vardecl_stmt> ; | <assign_stmt> ; | <compound_assign_stmt> ;
        | <expr> ; | <if_stmt> | <while_stmt> | <do_while_stmt> |
```

```
<for_stmt>
        | <switch_stmt> | <break_stmt> ; | <continue_stmt> ;
        | <printf_stmt> ; | <scanf_stmt> ; | <return_stmt> ; | <block> |
;
<type_spec> ::= int | float | double | char | long
             | long int | long long | long long int
             | unsigned long | unsigned long long
<declarator> ::= <id> | <id> = <expr>
<declarator_list> ::= <declarator> | <declarator_list> , <declarator>
<vardecl_stmt> ::= <type_spec> <declarator_list>
                | const <type_spec> <declarator_list>
<assign_stmt> ::= <id> = <expr>
<compound_assign_stmt> ::= <id> += <expr> | <id> -= <expr>
                        | <id> *= <expr> | <id> /= <expr> | <id> %=
<expr>
<if_stmt> ::= if ( <expr> ) <block>
           | if ( <expr> ) <block> else <block>
           | if ( <expr> ) <block> else <if_stmt>
<while_stmt> ::= while ( <expr> ) <block>
<do_while_stmt> ::= do <block> while ( <expr> )
<for_stmt> ::= for ( <for_init> ; <for_cond> ; <for_update> ) <block>
<for_init> ::= ε | <vardecl> | <assign_stmt>
<for_cond> ::= ε | <expr>
<for_update> ::= ε | <assign_stmt> | <compound_assign_stmt> | <expr>
<switch_stmt> ::= switch ( <expr> ) { <case_list> }
<case_list> ::= ε | <case_list> <case_clause>
<case_clause> ::= case <case_value> : <stmt_list> | default : <stmt_list>
<case_value> ::= <int_lit> | <char_lit>
<break_stmt> ::= break
<continue_stmt> ::= continue
<printf_stmt> ::= printf ( <string_lit> <printf_arglist> )
<printf_arglist> ::= ε | , <expr> <printf_arglist>
<scanf_stmt> ::= scanf ( <string_lit> <scanf_arglist> )
<scanf_arglist> ::= ε | , & <id> <scanf_arglist>
```

```
<return_stmt> ::= return | return <expr>
<expr> ::= <int_lit> | <float_lit> | <char_lit> | <string_lit> | <id>
        | ( <expr> ) | - <expr> | ! <expr> | ~ <expr>
        | ++ <id> | -- <id> | <id> ++ | <id> --
        | <expr> + <expr> | <expr> - <expr> | <expr> * <expr>
        | <expr> / <expr> | <expr> % <expr>
        | <expr> < <expr> | <expr> > <expr> | <expr> <= <expr> | <expr>
>= <expr>
        | <expr> == <expr> | <expr> != <expr>
        | <expr> && <expr> | <expr> || <expr>
        | <expr> & <expr> | <expr> | <expr> | <expr> ^ <expr>
        | <expr> << <expr> | <expr> >> <expr>
        | <expr> ? <expr> : <expr>
        | sizeof ( <type_spec> ) | sizeof ( <expr> )
        | sqrt ( <expr> ) | pow ( <expr> , <expr> )
        | abs ( <expr> ) | ceil ( <expr> ) | floor ( <expr> )
```

3.5.2 C++ CFG:

```
<program> ::= int <id> ( ) <block>
<block> ::= { <stmt_list> }
<stmt_list> ::= ε | <stmt_list> <stmt>
<stmt> ::= <vardecl> ; | <assign_stmt> ; | <compound_assign_stmt> ;
        | <expr> ; | <if_stmt> | <while_stmt> | <do_while_stmt> |
<for_stmt>
        | <switch_stmt> | break ; | continue ; | <return_stmt> ;
        | <block> | ;
<type_spec> ::= int | float | double | char | bool | string
             | long | long int | long long
             | unsigned | unsigned int | unsigned long | unsigned long long
<declarator> ::= <id> | <id> = <expr>
<declarator_list> ::= <declarator> | <declarator_list> , <declarator>
<vardecl> ::= <type_spec> <declarator_list>
           | const <type_spec> <declarator_list>
<assign_stmt> ::= <id> = <expr>
```

```
<compound_assign_stmt> ::= <id> += <expr> | <id> -= <expr>
                         | <id> *= <expr> | <id> /= <expr> | <id> %= <expr>
<if_stmt> ::= if ( <expr> ) <block>
           | if ( <expr> ) <block> else <block>
           | if ( <expr> ) <block> else <if_stmt>
<while_stmt> ::= while ( <expr> ) <block>
<do_while_stmt> ::= do <block> while ( <expr> ) ;
<for_stmt> ::= for ( <for_init> ; <for_cond> ; <for_update> ) <block>
<for_init> ::= ε | <vardecl> | <assign_stmt>
<for_cond> ::= ε | <expr>
<for_update> ::= ε | <assign_stmt> | <compound_assign_stmt> | <expr>
<switch_stmt> ::= switch ( <expr> ) { <case_list> }
<case_list> ::= ε | <case_list> <case_stmt>
<case_stmt> ::= case <expr> : <stmt_list> | default : <stmt_list>
<return_stmt> ::= return | return <expr>
<expr> ::= <int_lit> | <float_lit> | <char_lit> | <string_lit>
        | true | false | cout | cin | endl | <id> | ( <expr> )
        | - <expr> | ! <expr> | ~ <expr>
        | ++ <id> | -- <id> | <id> ++ | <id> --
        | <expr> + <expr> | <expr> - <expr> | <expr> * <expr>
        | <expr> / <expr> | <expr> % <expr>
        | <expr> < <expr> | <expr> > <expr> | <expr> <= <expr> | <expr> >=
<expr>
        | <expr> == <expr> | <expr> != <expr>
        | <expr> && <expr> | <expr> || <expr>
        | <expr> & <expr> | <expr> | <expr> | <expr> ^ <expr>
        | <expr> << <expr> | <expr> >> <expr>
        | <expr> ? <expr> : <expr>
        | sizeof ( <type_spec> ) | sizeof ( <id> )
```

The C++ grammar treats stream insertion/extraction as ordinary binary operators.

3.5.3 Java CFG:

```
<program> ::= <import_list> public class <id> {
                public static void main ( String [ ] <id> ) <block> }
```

```
<import_list> ::= ε | <import_list> <import_decl>
<import_decl> ::= import <dotted_name> ;
<dotted_name> ::= <id> | <dotted_name> . <id>
                | <dotted_name> . Scanner | <dotted_name> . *
<block> ::= { <stmt_list> }
<stmt_list> ::= ε | <stmt_list> <stmt>
<stmt> ::= <vardecl> ; | <assign_stmt> ; | <compound_assign_stmt> ;
        | <expr> ; | <if_stmt> | <while_stmt> | <do_while_stmt> ;
        | <for_stmt> | <switch_stmt> | break ; | continue ;
        | <return_stmt> ; | <print_stmt> | <block> | ;
<type_spec> ::= int | float | double | char | boolean | String | Scanner
<vardecl> ::= <type_spec> <id>
           | <type_spec> <id> = <expr>
           | final <type_spec> <id>
           | final <type_spec> <id> = <expr>
<assign_stmt> ::= <id> = <expr>
<compound_assign_stmt> ::= <id> += <expr> | <id> -= <expr>
                         | <id> *= <expr> | <id> /= <expr> | <id> %= <expr>
<if_stmt> ::= if ( <expr> ) <block>
           | if ( <expr> ) <block> else <block>
           | if ( <expr> ) <block> else <if_stmt>
<while_stmt> ::= while ( <expr> ) <block>
<for_stmt> ::= for ( <for_init> ; <for_cond> ; <for_update> ) <block>
<for_init> ::= ε | <vardecl> | <assign_stmt>
<for_cond> ::= ε | <expr>
<for_update> ::= ε | <assign_stmt> | <compound_assign_stmt> | <expr>
<do_while_stmt> ::= do <block> while ( <expr> )
<switch_stmt> ::= switch ( <expr> ) { <case_clauses> }
<case_clauses> ::= ε | <case_clauses> <case_clause>
<case_clause> ::= case <int_lit> : <case_body>
                | case <char_lit> : <case_body>
                | case <string_lit> : <case_body>
                | default : <case_body>
<case_body> ::= ε | <case_body> <stmt>
```

```
<return_stmt> ::= return | return <expr>
<print_stmt> ::= System.out.println ( )
               | System.out.println ( <expr> )
               | System.out.print ( <expr> )
<primary_expr> ::= <int_lit> | <float_lit> | <char_lit> | <string_lit>
                 | true | false | System.in | <id> | <new_expr> | ( <expr>
)
<new_expr> ::= new Scanner ( <arg_list_opt> )
<arg_list_opt> ::= ε | <arg_list>
<arg_list> ::= <expr> | <arg_list> , <expr>
<postfix_expr> ::= <primary_expr>
                 | <postfix_expr> . <id> ( <arg_list_opt> )
<expr> ::= <postfix_expr> | - <expr> | ! <expr> | ~ <expr>
        | ++ <id> | -- <id> | <id> ++ | <id> --
        | <expr> + <expr> | <expr> - <expr> | <expr> * <expr>
        | <expr> / <expr> | <expr> % <expr>
        | <expr> < <expr> | <expr> > <expr> | <expr> <= <expr> | <expr> >=
<expr>
        | <expr> == <expr> | <expr> != <expr>
        | <expr> && <expr> | <expr> || <expr>
        | <expr> & <expr> | <expr> | <expr> ^ <expr>
        | <expr> << <expr> | <expr> >> <expr>
        | <expr> ? <expr> : <expr>
```

The Java parser is intentionally a small subset of Java syntax. It requires the outer class/main structure and recognizes imports, but import nodes have no runtime effect. Scanner and selected String methods are interpreted by dedicated AST nodes and interpreter logic.

4. Compiler Architecture

The core architecture is a sequence of compiler phases.

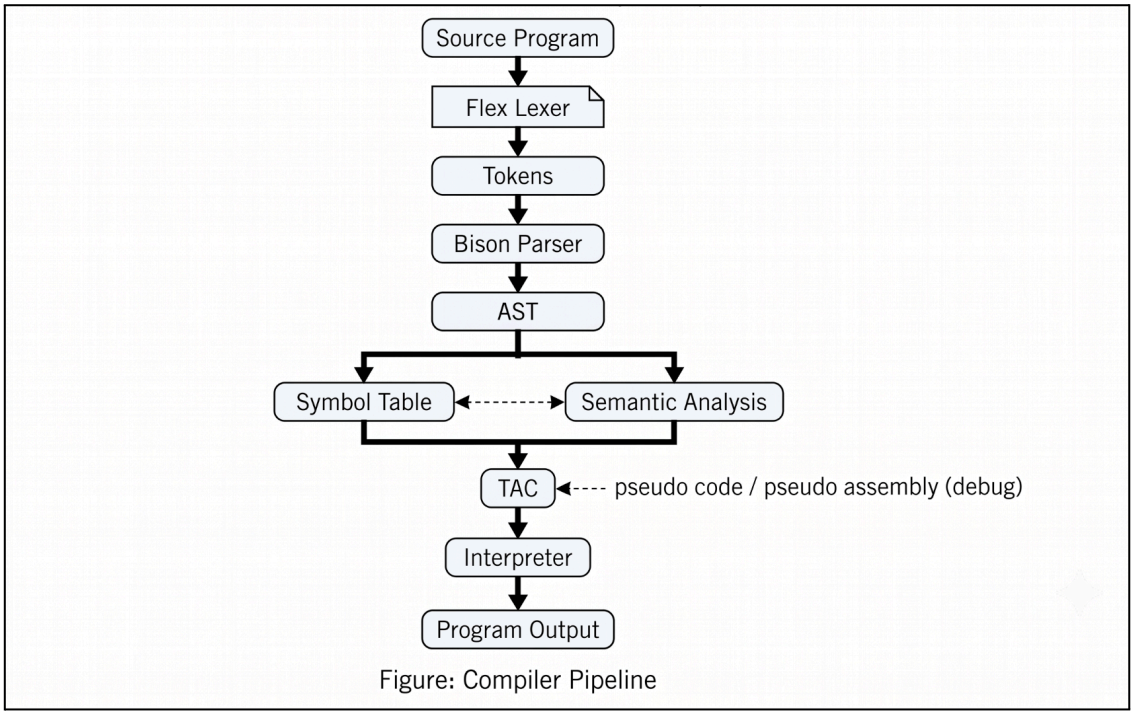


Figure: Compiler Pipeline

4.1 Module Communication

Module	Input	Processing	Output	Next consumer
lexer.l	Source characters	Pattern matching and tokenization	Token stream	parser.y
parser.y	Tokens	CFG recognition and AST actions	AST root	semantic.cpp / tac.cpp / interpreter.cpp
ast.h / ast.cpp	Parser-created nodes	Node representation and debug printing	AST structure	all AST walkers
semantic.cpp	AST	Name, declaration, control-flow and selected semantic checks	Error list / success	main.cpp
symbol_table.cpp	Declarations/lookups	Scope stack and symbol storage	Symbol entries	semantic.cpp / interpreter.cpp
tac.cpp	AST	Temporary/label lowering	TAC lines	debug display
interpreter.cpp	AST + runtime table	Tree-walking execution	Program output	user/frontend
main.cpp	Input	Phase control and stopping on errors	Compiler/debug output	user/frontend

4.2 Driver Control

main.cpp controls the full sequence. In debug mode it first runs the scanner to print tokens, rewinds the file, and calls yyparse() for the actual parse. If parsing succeeds, semantic analysis is run. If semantic analysis succeeds, debug mode may print TAC, pseudo target code, and pseudo assembly. Finally, the interpreter executes the AST using a fresh runtime symbol table.

4.3 Frontend and Backend Support

The browser frontend and Node.js backend are supporting components. They select C, C++, or Java, send source text to the corresponding compiler executable, and display the compiler's output. They do not replace any compiler phase.

5. Lexer Design

Lexical analysis is the first compiler phase. Flex reads source characters and matches them against ordered regular expressions. The action attached to a rule returns a Bison token or discards the text.

5.1 Token Categories

Category	Examples
Keywords	int, float, double, char, bool/boolean, String/string, if, else, while, for, do, switch, break, continue, return
Identifiers	letter or underscore followed by letters, digits, or underscores
Literals	integer, floating-point, character, string, boolean
Operators	+ - * / %, relational, logical, bitwise, shifts, assignments, ++/--, ?:
Delimiters	parentheses, braces, semicolon, comma; Java also brackets and dot
Comments	// and /*...*/
Special constructs	printf/scanf, cout/cin/endl, System.out.*, Scanner/System.in

5.2 Regular Expressions Used in the Lexer

Token	Regular Expression / Pattern	Example
Identifier	`[a-zA-Z_][a-zA-Z0-9_]*`	`count`, `_value1`
Integer	`[0-9]+`	`10`, `250`

Floating-point	`[0-9]+\.[0-9]+`	`3.14`, `10.5`
Character	`'([^'\]`	`\.)'`
String	`"([^"\]`	`\.)*"`
Whitespace	`[ \t\r\n]+`	spaces/newlines
Single-line comment	`//.*`	`// comment`
Multi-line comment	`/*([^*]`	`*+[^/])*+/`

5.3 Important Flex Decisions

The lexer uses %option yylineno for line tracking. Keyword rules appear before the generic identifier rule. Multi-character operators such as ==, <=, >=, &&, ||, <<, >>, +=, -=, *=, /=, %=, ++, and -- are defined explicitly so they are recognized as single tokens. String and character rules also convert basic escape sequences before placing their values into yylval.

5.4 Ignored Text and Errors

Comments and whitespace are discarded. C and C++ preprocessor-style lines beginning with # are skipped rather than parsed. C++ also skips using namespace std;. Unsupported characters produce lexical diagnostics. The supplied test programs use @ and $ as invalid-character cases.

6. Parser Design

Bison consumes the token stream and checks it against the CFG. The semantic actions in parser.y directly construct ASTNode objects, so the parser output is already suitable for later passes.

6.1 Grammar Used in the Parser

The grammar defines how tokens can be combined to form valid programs.

The parser contains grammar rules for the complete program structure, declarations, assignments, expressions, conditional statements, loops, switch statements, and return statements.

```
program
    → int identifier ( ) block
block
    → { statement_list }
statement_list
    → ε
    | statement_list statement
```

The Java parser has a different outer structure because it requires a Java class and main method.

```
program:
    → import_list public class identifier
      { public static void main ( String [ ] identifier )
      block }
```

6.1.1 Statement Grammar

The parsers recognize several types of statements. A simplified representation is:

```
statement
    → variable_declaration ;
    | assignment ;
    | compound_assignment ;
    | expression ;
    | if_statement
    | while_statement
    | do_while_statement
    | for_statement
    | switch_statement
    | break ;
    | continue ;
    | return ;
    | block
```

The exact available statements differ slightly between the C, C++, and Java implementations because each parser contains language-specific features.

6.1.2 Conditional Statements

The parsers contain the following structure for if statements:

```
if_stmt
    → if ( expr ) block
```

```
    | if ( expr ) block else block
| if ( expr ) block else if_stmt
```

This allows the compiler to recognize a simple if statement, an if-else statement, and nested else-if structures. When the parser recognizes an if statement, it creates a NODE_IF AST node. The condition becomes the first child, the then block becomes the second child, and the optional else part becomes the third child.

6.2 Expression Grammar

Expressions are one of the most important parts of the parser because many different operators can appear in an expression. The parsers support arithmetic, relational, logical, bitwise, shift, assignment-related, unary, and conditional expressions depending on the language implementation. The parser's semantic action then creates the corresponding NODE_BINOP AST nodes. The expression grammar uses a common expr non-terminal, while Bison precedence declarations determine how operators are grouped.

6.3 Precedence and Associativity Rules

The parser uses Bison's precedence and associativity declarations to correctly interpret expressions containing multiple operators.

Precedence Level	Operators	Associativity
Lowest	? :	Right
	||	Left
	&&	Left
	|	Left
	^	Left
	&	Left
	==, !=	Left
	<, >, <=, >=	Non-associative
	<<, >>	Left
	+, -	Left
	*, /, %	Left
Highest	Unary operators	Non-associative

6.4 Conflict Resolution

Grammar conflicts occur when Bison can find more than one possible parsing action for the same input situation.

Bison uses **precedence and associativity rules** to resolve ambiguities between operators. Unary operators such as -a are given separate precedence using %prec UMINUS, while the recursive if-else grammar explicitly defines how else-if structures are parsed. The parser therefore produces the intended AST structure without relying on an unspecified interpretation.

6.5 AST Construction

After recognizing a valid grammar rule, the parser's semantic action creates the corresponding AST node. For example, if statements create NODE_IF, binary expressions create NODE_BINOP, and assignments create NODE_ASSIGN. The resulting AST is then passed to semantic analysis, TAC generation, and the interpreter.

6.6 Parser Error Handling

When the token sequence does not match the grammar, Bison calls yyerror() and reports a syntax error. Thus, the parser acts as the stage that converts the lexer's token stream into either a valid AST or a syntax-error result.

7. Abstract Syntax Tree (AST)

The AST is the main shared representation between parsing and the later compiler phases. It removes punctuation that has no meaning after parsing while preserving declarations, expressions, statements, and control flow.

7.1 Tagged ASTNode Design

Each compiler uses one ASTNode class with a NodeType enum.

Node	**Meaning**	**Children**
NODE_PROGRAM	Whole program	Ordered statements
NODE_BLOCK	Statement sequence	Statements
NODE_VARDECL	Variable declaration	Optional initializer
NODE_ASSIGN	Assignment	Expression
NODE_BINOP	Binary expression	Left, right
NODE_UNOP	Unary expression	Operand
NODE_TERNARY	Conditional expression	Condition, true, false
NODE_IF	Conditional statement	Condition, then, else

NODE_WHILE / NODE_DOWHILE	Loops	Condition/body as defined in each AST
NODE_FOR	For loop	Init, condition, update, body
NODE_SWITCH	Switch	Selector and cases
NODE_BREAK / NODE_CONTINUE	Loop/switch control	None
NODE_RETURN	Return	Optional expression

7.2 AST Construction Strategy

The AST is constructed during parsing using Bison semantic actions. When the parser recognizes a grammar rule, its action creates the appropriate AST node and connects its child nodes according to the structure of the statement or expression. For example, an `if` statement creates a `NODE_IF` with children for the condition, then-block, and optional else-block, while expressions create `NODE_BINOP` nodes with left and right operands. The completed AST is then passed to semantic analysis, TAC generation, and the interpreter.

.8. Semantic Analysis

Semantic analysis is performed after parsing and before interpretation. It walks the AST and checks properties that syntax alone cannot guarantee.

Semantic Rule Enforced	**How It Is Detected in the Compiler**
Undeclared variables	lookup() must find every referenced identifier.
Duplicate declarations	declare() rejects duplicate names in the current active scope.
Const/final modification	Assignments and increment/decrement are rejected when the symbol is const/final.
Division/modulo by literal zero	Literal zero right operands are detected.
break/continue context	loopDepth and switchDepth are tracked.
C bitwise restrictions	C rejects float/double operands for bitwise operations.
C++ cin target	Right side of cin >> must be a declared variable.
Java print restriction	print() needs an argument; println() may be empty.

9. Symbol Table

The symbol table is a data structure used by the compiler to store information about identifiers declared in the program. During semantic analysis, the compiler uses the symbol table to determine whether variables exist, whether a name has already been declared, and what properties are associated with each identifier.

9.1 Data Structure Design

Each symbol stores information needed by the semantic analyzer, such as the identifier name, data type, and declaration properties such as `const` or `final` where applicable. The symbol table provides operations such as:

- `declare()` — adds a newly declared identifier.
- `lookup()` — searches for an identifier when it is referenced.
- Update operations — maintain the stored information when required by the compiler.

The symbol table is therefore shared by semantic analysis and is used whenever the compiler needs information about an identifier.

9.2 Scope-Handling Strategy

The compiler handles scope by keeping track of the currently active declarations. When a variable is declared, `declare()` checks the active scope before inserting it. This prevents duplicate declarations within the same scope. When an identifier is used, `lookup()` searches the available symbol information to determine whether the variable has been declared. If no matching symbol is found, the semantic analyzer reports an undeclared-variable error. The symbol table also stores properties needed for semantic checks. For example, the semantic analyzer can determine whether an identifier is `const` or `final` before allowing an assignment or increment/decrement operation.

10. Intermediate Code Generation

TAC converts complex source constructs into simple linear operations. The project uses temporary variables t1, t2, ... and labels L1, L2, ...

```
Source:
   c = a + b * 2;
TAC:
```

```
t1 = b * 2
t2 = a + t1
c = t2
```

10.1 Control Flow

```
if (x > 0) { y = 1; } else { y = 2; }
t1 = x > 0
if t1 == 0 goto L1
y = 1
goto L2
L1:
y = 2
L2:
```

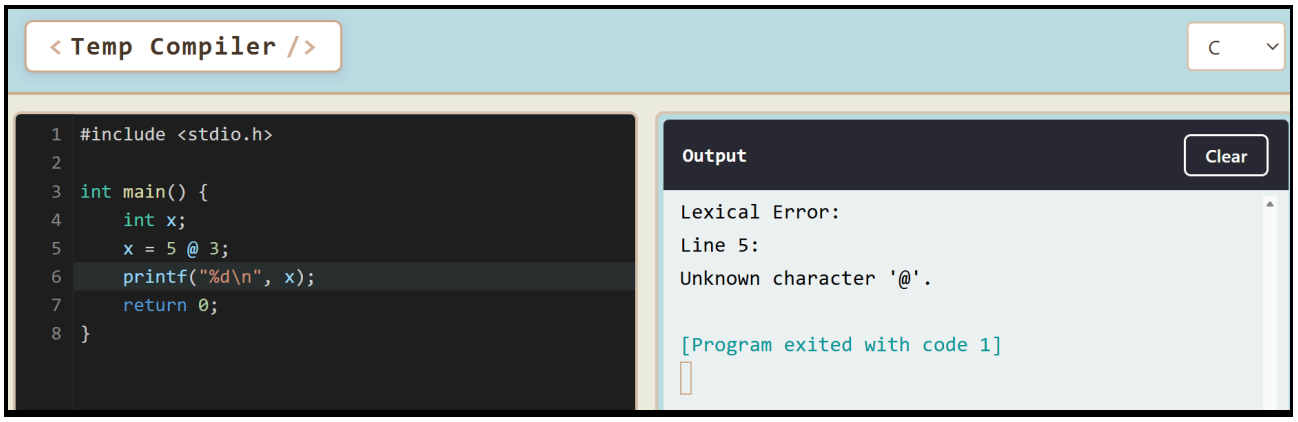
C++ and Java generators also use label pairs to lower loop break/continue, and the C++ generator lowers ternary expressions with labels. However, TAC is not the execution engine and is not complete for every parsed construct in every language.

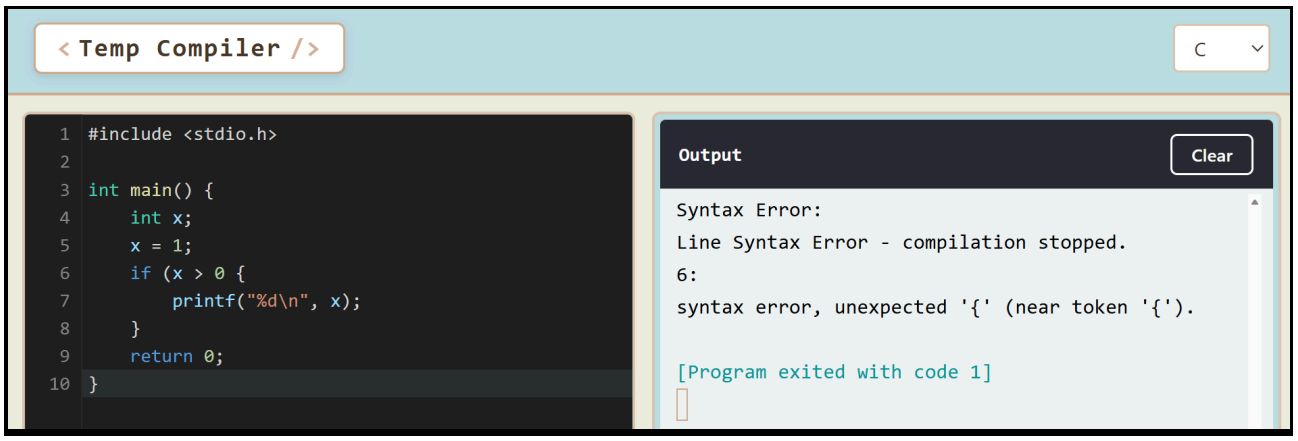
11. Challenges

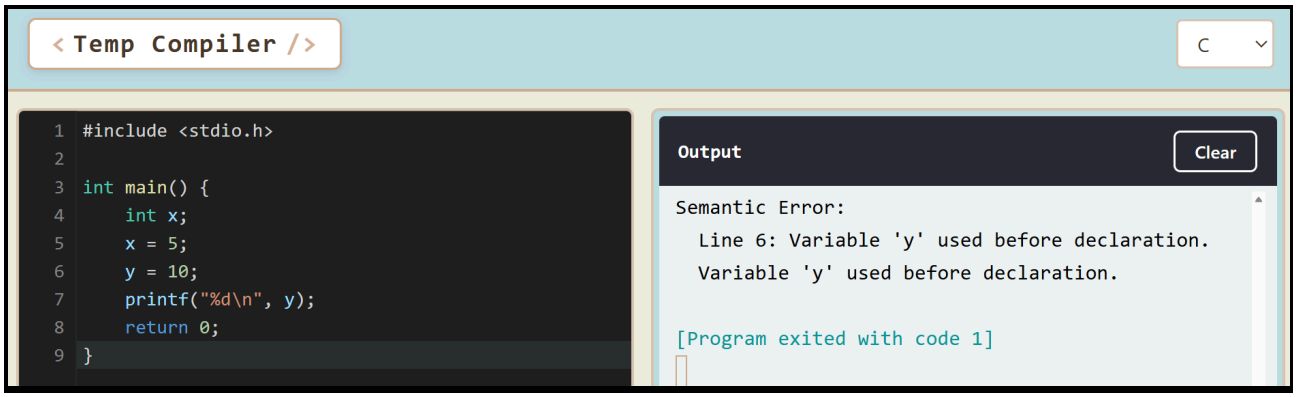
- **Flex/Bison C++ Integration:** Integrating C++ pointers between Flex and Bison was difficult.Generated C++ scanner/parser files and compiled them using C++17.
- **AST Consistency:** Maintaining a consistent AST across three language implementations was challenging.Used a tagged `NodeType`-based AST structure.
- **Semantic and Control-Flow Validation:** Detecting undeclared variables and invalid `break`/`continue` usage required additional context. Used `SymbolTable` lookup/declaration functions and tracked `loopDepth` and `switchDepth`.
- **Language-Specific Features and Execution:** C, C++, and Java have different syntax and I/O models, while TAC coverage is partial. Used language-specific tokens/AST nodes and kept the interpreter as the actual execution path.

12. Testing

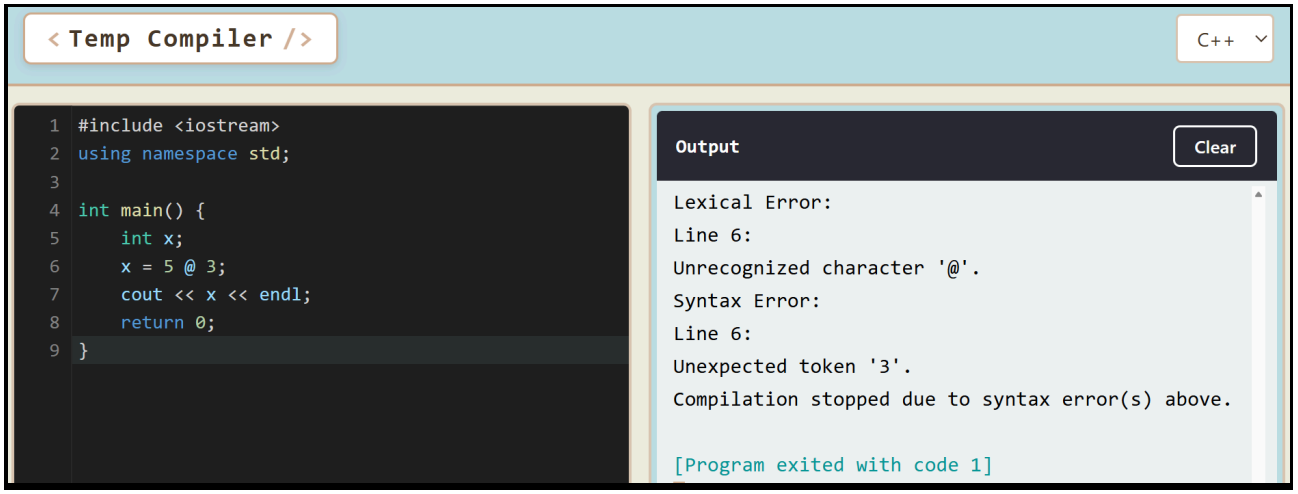
12.1 C Language - Lexical,Syntax,Semantic error examples:

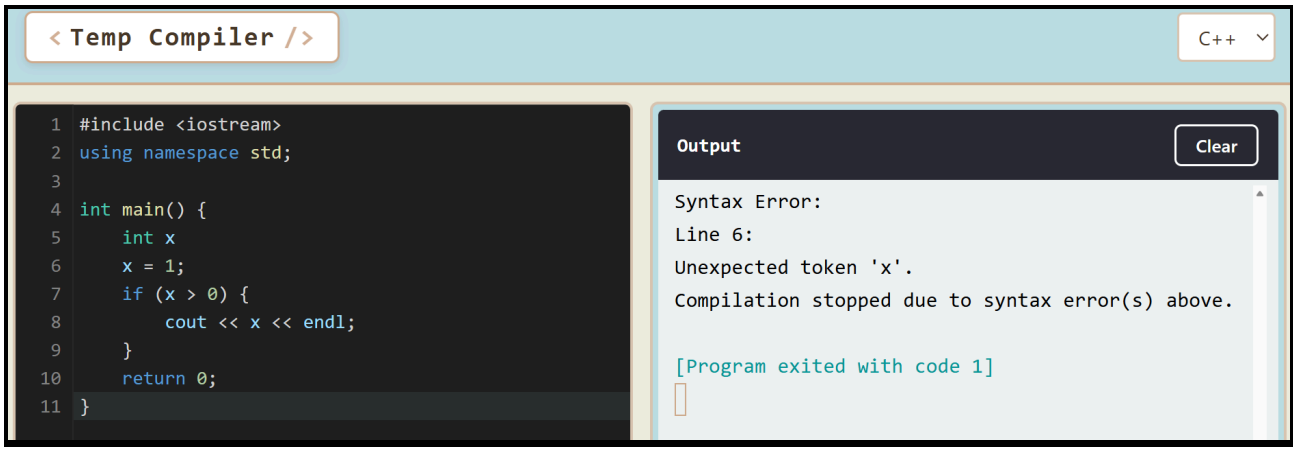


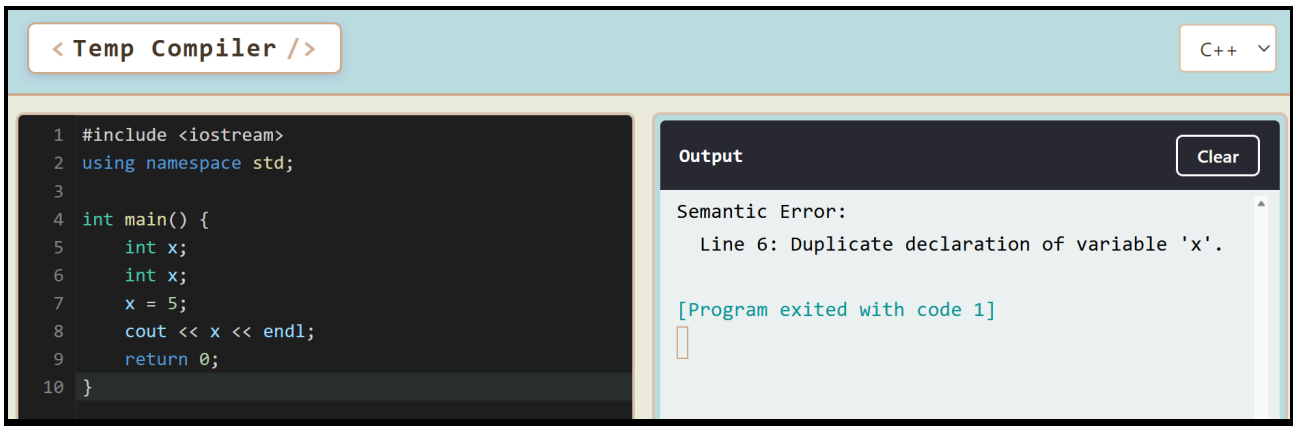




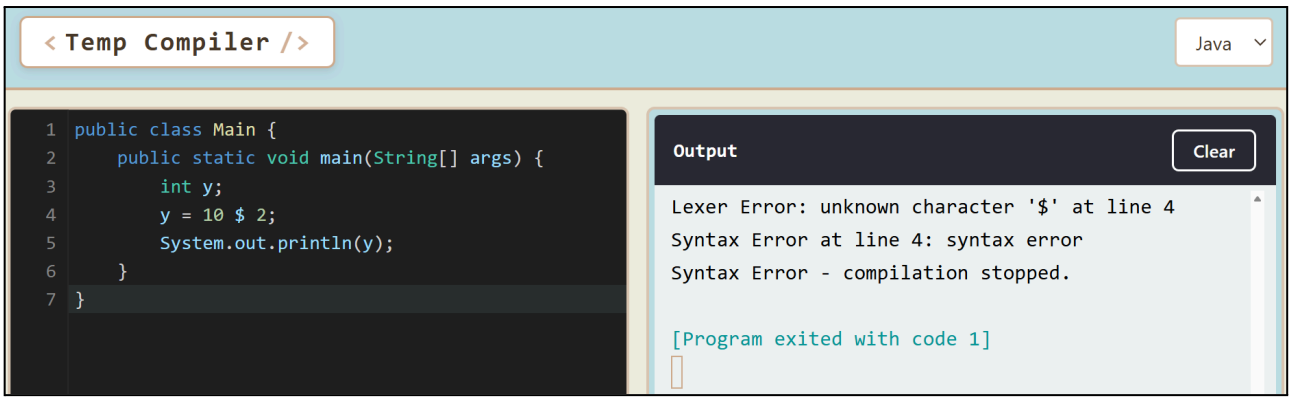
12.2 CPP Language - Lexical,Syntax,Semantic error examples:

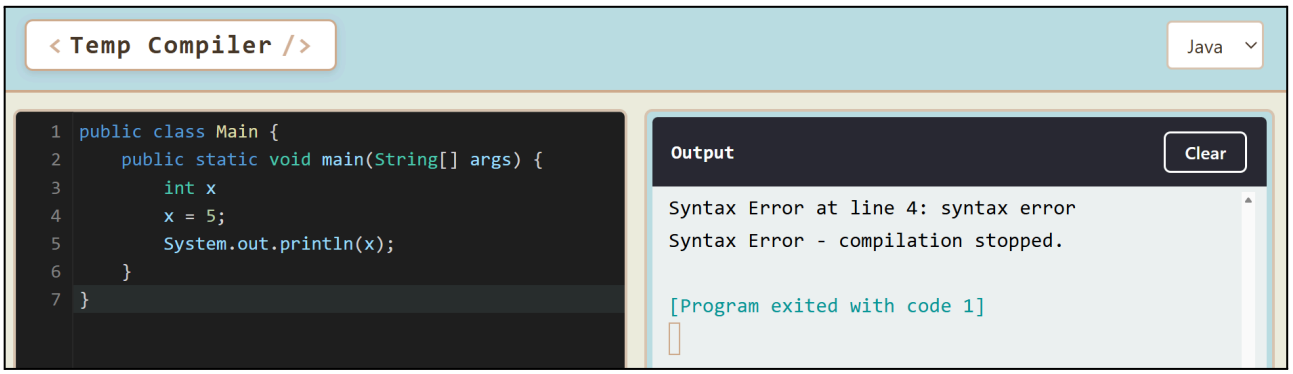


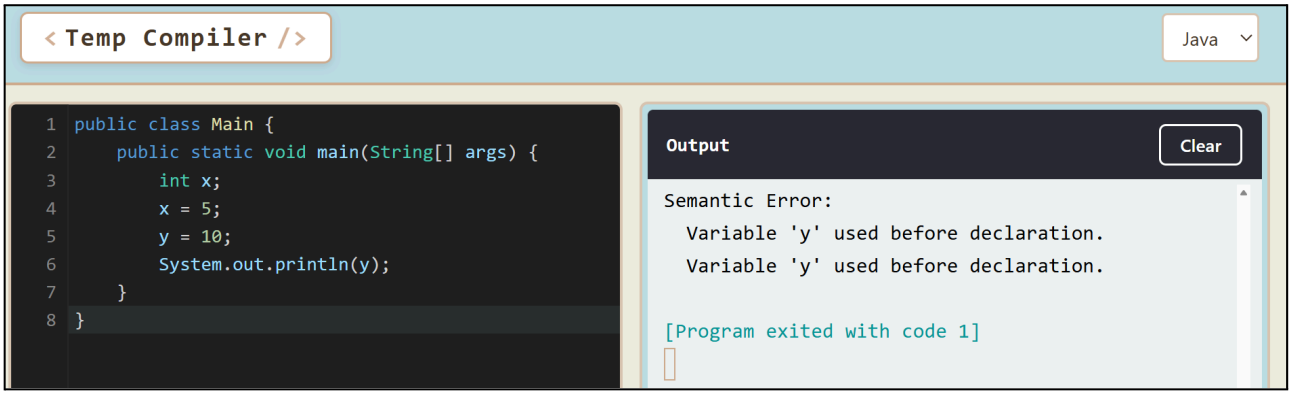




12.3 Java Language - Lexical,Syntax,Semantic error examples:







12.4 Valid Program Example

- **C Language:**

- **CPP Language:**

- **Java Language:**

13. Conclusion

The project successfully demonstrates the main ideas of compiler construction through a modular Flex/Bison architecture. The lexer turns source characters into tokens, the parser checks those tokens against a CFG and builds an AST, semantic analysis validates important program rules, the symbol table provides name information, TAC provides a linear intermediate representation, and the interpreter executes the validated AST.

The AST-centered design is the strongest architectural feature. It allows several independent passes to consume the same structured program representation. This is why the project can support both interpretation and intermediate-code visualization without changing the lexer.

13.1 Limitations

- Scope creation is mainly for main and for-loop scopes, not every nested block.
- TAC does not lower every construct supported by the parsers/interpreters.
- Pseudo code and assembly are debug output, not a real hardware backend.
- Arrays, pointers, user-defined functions, full classes, linking, and executable generation are outside the implemented subset.

14.References

[1] J. Smith, *From Source Code to Machine Code: Build Your Own Compiler From Scratch*. build-your-own.org, May 18, 2023.